

Capital University

Faculty of Computing & Artificial Intelligence

Wiggle Sort

Algorithm Project

Prepared by: Mega Team

Table of Contents

1. Introduction	2
2. First algorithm (Recursive)	2
2.1 pseudocode:	2
2.2 implementation:.....	3
2.3 Analysis:.....	3
2.4 Complexity:.....	4
2.4.1 Time	4
2.4.2 Space	4
3. Second algorithm (non-recursive):.....	4
3.1 pseudocode:	4
3.2 Implementation:	4
3.3 Analysis:.....	5
3.4 Complexity:.....	5
3.4.1 Time:	5
3.4.2 Space:	5
4. Comparison:.....	6

1. Introduction

In this project, we implement and analyze a specific variation of sorting known as Wiggle Sort.

The Wiggle Sort algorithm rearranges elements in an array such that the elements follow an alternating pattern:

Elements at even indices are less than or equal to their next element.

Elements at odd indices are greater than or equal to their next element.

This project presents two different implementations of the Wiggle Sort algorithm:

A recursive approach, which uses function calls to achieve the desired ordering.

A non-recursive (iterative) approach, which uses loops for direct execution.

Both implementations are analyzed in terms of time complexity, space complexity, efficiency, and overall performance. The main objective of this project is to compare recursion and iteration in solving the same problem and understand the trade-offs between them in terms of memory usage and execution efficiency.

2. First algorithm (Recursive)

2.1 pseudocode:

```
function swap(i, j)
```

```
    temp ← value at i
```

```
    value at i ← value at j
```

```
    value at j ← temp
```

```
function wigglesort(arr, n, i)
```

```
    if  $i \geq n - 1$  then
```

```
        return
```

```
    if  $i \bmod 2 = 0$  and  $\text{arr}[i] > \text{arr}[i + 1]$  then
```

```
        swap(arr[i], arr[i + 1])
```

```
if i mod 2 ≠ 0 and arr[i] < arr[i + 1] then  
    swap(arr[i], arr[i + 1])
```

```
wigglesort(arr, n, i + 1)
```

2.2 implementation:

```
void swap(int *i, int *j){  
    int temp = *i;  
    *i = *j;  
    *j = temp;  
}  
  
void wiggleSort(int arr[], int n , int i){  
    if(i >= n-1)  
        return;  
    if(i %2 == 0 && arr[i]>arr[i+1] )  
        swap(&arr[i],&arr[i+1]);  
  
    if (i %2 != 0 && arr[i]<arr[i+1] )  
        swap(&arr[i],&arr[i+1]);  
  
    wiggleSort(arr, n, i+1);  
}
```

2.3 Analysis:

Start from index i

If $i \geq n - 1$, stop recursion

For each index:

If i is even and $arr[i] > arr[i+1] \rightarrow$ swap

If i is odd and $arr[i] < arr[i+1] \rightarrow$ swap

Recursively call the function for $i + 1$

2.4 Complexity:

2.4.1 Time

$$T(n) = T(n-1) + O(1)$$

$$T(n) = O(1) + O(1) + O(1) + \dots + O(1) \text{ (n times)}$$

$$\text{Time complexity} = O(n)$$

- Best case : $O(n)$
- Average case : $O(n)$
- Worst case : $O(n)$

2.4.2 Space

$$\text{Space complexity} = O(n)$$

3. Second algorithm (non-recursive):

3.1 pseudocode:

```
function swap(i, j)
    temp ← value at i
    value at i ← value at j
    value at j ← temp
function wigglesort(arr, n)
    for i ← 0 to n - 2 do

        if i mod 2 = 0 and arr[i] > arr[i + 1] then
            swap(arr[i], arr[i + 1])

        if i mod 2 ≠ 0 and arr[i] < arr[i + 1] then
            swap(arr[i], arr[i + 1])
```

3.2 Implementation:

```
void swap(int *i, int *j){
    int temp = *i;
```

```

        *i = *j;
        *j = temp;
    }

    void wiggleSort(int arr[], int n){
        for( i = 0 ; i < n-1 ; i++){

            if(i %2 == 0 && arr[i] > arr[i+1])
                swap(&arr[i],&arr[i+1]);

            if(i %2 != 0 && arr[i] < arr[i+1])
                swap(&arr[i],&arr[i+1]);
        }
    }

```

3.3 Analysis:

At even indices ($i \% 2 == 0$):

→ $arr[i] \leq arr[i + 1]$

At odd indices ($i \% 2 != 0$):

→ $arr[i] \geq arr[i + 1]$

The algorithm works by:

Traversing the array once from i to $n - 2$

Comparing adjacent elements

Swapping them if they do not satisfy the wiggle condition

3.4 Complexity:

3.4.1 Time:

the loop runs from 0 to $n - 2$

so the number of iterations is proportional to n

Time complexity = $O(n)$

- Best case : $O(n)$
- Average case : $O(n)$
- Worst case : $O(n)$

3.4.2 Space:

Space complexity = $O(1)$

4. Comparison:

Feature	Recursive	Non-Recursive
Approach	Uses function calls	Uses loop
Control Flow	Call stack-based execution	Linear Execution
Time Complexity	$O(n)$	$O(n)$
Space Complexity	$O(n)$	$O(1)$
Efficiency	Less efficient	More efficient
Risk	Possible stack overflow for large n	No Stack overflow
Readability	More elegant but slightly harder to trace	Simple & direct